# Embedded C and CMSIS

*Introduction to ARM Cortex-M Microprocessors* — Chapter 4

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

# Table of Contents


1. Why This Matters
2. What the Compiler Does
3. Why Embedded C Rather Than Assembly
4. The CMSIS Framework
5. The Three Pillars of CMSIS Register Access
6. The volatile Keyword
7. Bit Manipulation on Peripheral Registers
8. A Complete Bare-Metal Program
9. What Happens When the STM32 Starts Up
10. Summary

# Why This Chapter Matters

This chapter joins the two halves of the book. You know C. You know how the processor and memory are organized. Now we connect them: a C variable becomes a register, a C structure becomes a peripheral, and one assignment becomes a voltage on a pin. This is where hardware first moves.

## After this chapter, you can:

1. Predict the assembly the compiler generates for simple C.
2. Explain the CMSIS components and which one does register-level work.
3. Show how a C structure overlays a peripheral's registers.
4. Justify `volatile` with two optimizations it prevents.
5. Apply the set, clear, toggle and test idioms.
6. Write a bare-metal CMSIS program that drives a GPIO pin.

© Muním Zabidi

By the end of this chapter you will know exactly how this one line of C changes a voltage on a physical pin:

```
GPIOA->ODR ^= (1U << 5);    // toggle the user LED on PA5
```

Questions this code raises, and where this chapter answers them:

| Question raised | Answered in |
|---|---|
| What is `GPIOA->ODR`? | CMSIS: a struct laid over the registers |
| Why `(1U << 5)`? | Bit manipulation: building a mask |
| Why is nothing deleted by the compiler? | The `volatile` keyword |
| Why does nothing happen with the clock off? | A complete bare-metal program |

# Table of Contents


1 Why This Matters

2 What the Compiler Does

3 Why Embedded C Rather Than Assembly

4 The CMSIS Framework

5 The Three Pillars of CMSIS Register Access

6 The volatile Keyword

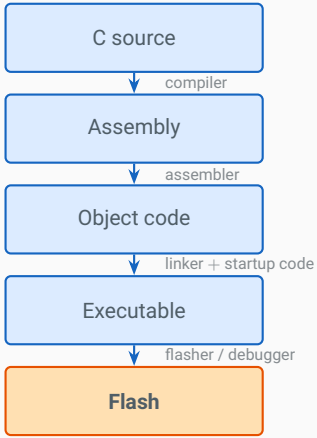
7 Bit Manipulation on Peripheral Registers

8 A Complete Bare-Metal Program

9 What Happens When the STM32 Starts Up

10 Summary

# What the Compiler Does




Each step is a separate program. You can inspect each one. This is how you find out what your C code actually became.

## Why you learned assembly first

You will write C for the rest of this book. But when something goes wrong, the debugger shows you instructions, not C.

Chapters 5–7 exist so this disassembly window is readable, not frightening.

The linker adds startup code. It sets the stack pointer, zeroes your global variables, and calls `main()`, all before any line you wrote runs.

© Munim Zabidi

# A C Statement, Compiled What the compiler writes for you

The Cortex-M4 is a load-store machine, so arithmetic happens only in registers. The compiler must move the values in and out.

```c
int x = 5;
int y = 10;
int z = x + y;
```

**Notice**

- Locals live on the stack, as offsets from SP.
- Arithmetic uses registers only.
- With optimisation, the compiler may remove memory accesses. This is why `volatile` exists.

```
MOV   R0, #5          ; x = 5
STR   R0, [SP, #0]    ; store x on the stack
MOV   R1, #10         ; y = 10
STR   R1, [SP, #4]    ; store y
LDR   R0, [SP, #0]    ; reload x
LDR   R1, [SP, #4]    ; reload y
ADD   R2, R0, R1      ; z = x + y
STR   R2, [SP, #8]    ; store z
```

A function is an idea in the C language. The processor has no built-in notion of one.

When one function calls another and later resumes, the compiler uses registers, memory and a calling convention. The convention fixes three things:

| The convention decides | On the Cortex-M4 |
|---|---|
| Which registers carry the arguments | `R0`–`R3` |
| Where the return value is placed | `R0` |
| How the return address is kept | `LR`, set by `BL` |

**C and assembly can mix**

If both follow the same convention, C can call a function written in assembly, and the reverse. Chapters 5–7 show how.

© Munim Zabidi

# Table of Contents


1 Why This Matters

2 What the Compiler Does

3 **Why Embedded C Rather Than Assembly**

4 The CMSIS Framework

5 The Three Pillars of CMSIS Register Access

6 The volatile Keyword

7 Bit Manipulation on Peripheral Registers

8 A Complete Bare-Metal Program

9 What Happens When the STM32 Starts Up

10 Summary

# C or Assembly?

| | C | Assembly |
|---|---|---|
| Development speed | Fast | Slow |
| Readability | Good | Poor |
| Portability | Across ARM families | None |
| Code size | Good, with a modern compiler | Smallest possible |
| Hardware access | Through pointers and CMSIS | Direct |
| Typical use | Almost everything | Startup, ISR entry, hand-tuned DSP |

## What C gives you, and what it does not

C gives you the same hardware access as assembly, with far less bookkeeping. What it does *not* give you is protection. There is no runtime check to catch a bad pointer.

| Principle | In practice |
|---|---|
| **Readability** | Use clear names and consistent indents. Comment on *why* a bit is set. The datasheet already says what it does. |
| **Modularity** | Small functions with one purpose. One function for each peripheral operation. |
| **No magic numbers** | Name your constants. `LED_PIN` is clearer than `(1U << 5)`. |
| **Explicit types** | Always use the fixed-width types from Chapter 3. |
| **Defensive checks** | Check ranges and returned status before you use a value. |

© Muním Zabidi

# Table of Contents


1 Why This Matters

2 What the Compiler Does

3 Why Embedded C Rather Than Assembly

4 **The CMSIS Framework**

5 The Three Pillars of CMSIS Register Access

6 The volatile Keyword

7 Bit Manipulation on Peripheral Registers

8 A Complete Bare-Metal Program

9 What Happens When the STM32 Starts Up

10 Summary

# What CMSIS Is

**C**ortex **M**icrocontroller **S**oftware **I**nterface **S**tandard. This is ARM's layer between your C code and the silicon. It works the same way across different chip vendors.

## What it gives you

- A struct per peripheral, laid out on the real register offsets
- A pointer to each peripheral at its base address
- Named constants for bit fields
- Core functions: `__enable_irq()`, `NVIC_EnableIRQ()`, `__WFI()`

You need just one header: `#include "stm32f446xx.h"`

## Why it matters

- The same names across every vendor's Cortex-M
- Readable: `GPIOA->ODR` rather than `*(volatile uint32_t*)0x40020014`
- The struct layout does the address arithmetic for you

# The CMSIS Components

CMSIS is ARM's standard layer. It names every register the same way on any vendor's Cortex-M chip. It has six parts:

| Component | What it gives you |
|---|---|
| CMSIS-CORE | Standard access to core registers, the NVIC, SysTick and all peripheral register structures. Used throughout this book. |
| CMSIS-DSP | Optimised filters, FFTs, matrix and vector maths. |
| CMSIS-RTOS | A standard API that keeps an application independent of the real-time kernel. |
| CMSIS-Driver | Portable peripheral driver interfaces for middleware. |
| CMSIS-NN | Efficient neural-network kernels for small devices. |
| CMSIS-SVD | XML descriptions of every register and bit field. This is how a debugger shows registers by name. |

© Muním Zabidi

# Table of Contents


1 Why This Matters

2 What the Compiler Does

3 Why Embedded C Rather Than Assembly

4 The CMSIS Framework

5 **The Three Pillars of CMSIS Register Access**

6 The volatile Keyword

7 Bit Manipulation on Peripheral Registers

8 A Complete Bare-Metal Program

9 What Happens When the STM32 Starts Up

10 Summary

# A Peripheral as a Struct

```
typedef struct {
  __IO uint32_t MODER;
  __IO uint32_t OTYPER;
  __IO uint32_t OSPEEDR;
  __IO uint32_t PUPDR;
  __IO uint32_t IDR;
  __IO uint32_t ODR;
  __IO uint32_t BSRR;
  __IO uint32_t LCKR;
  __IO uint32_t AFR[2];
} GPIO_TypeDef;
```

Each member is one 32-bit register, in hardware order. A `uint32_t` is 4 bytes, so member *n* sits at offset 4*n*. These are exactly the offsets given in the reference manual.

`__IO` expands to `volatile`. CMSIS marks every register for you.

`AFR[2]` uses Chapter 3's array indexing: two real registers, reached as `AFR[0]` and `AFR[1]`.

# Access Qualifiers: __I, __O, __IO

Each struct member carries a qualifier. It records what you may do with that register.

| Qualifier | Access | Defined as | Example |
|---|---|---|---|
| `__I` | read-only | `volatile const` | `IDR`: set by outside signals |
| `__O` | write-only | `volatile` | `BSRR`: a trigger |
| `__IO` | read/write | `volatile` | `MODER`, `ODR` |

**Every one of them includes `volatile`**

Writing to a read-only register is a compile-time error. The reason for `volatile` is explained in the next section.

© Muním Zabidi

The vendor defines each peripheral's base address as a macro, built up from the memory map of Chapter 2:

```
#define PERIPH_BASE      0x40000000UL                // Peripheral region
#define AHB1PERIPH_BASE  (PERIPH_BASE + 0x00020000UL) // AHB1 bus
#define GPIOA_BASE       (AHB1PERIPH_BASE + 0x0000UL) // Port A
#define GPIOC_BASE       (AHB1PERIPH_BASE + 0x0800UL) // Port C
```

| Macro | Value |
|---|---|
| GPIOA_BASE | 0x40000000 + 0x00020000 + 0x0000 = 0x40020000 |
| GPIOC_BASE | 0x40000000 + 0x00020000 + 0x0800 = 0x40020800 |

© Muním Zabidi

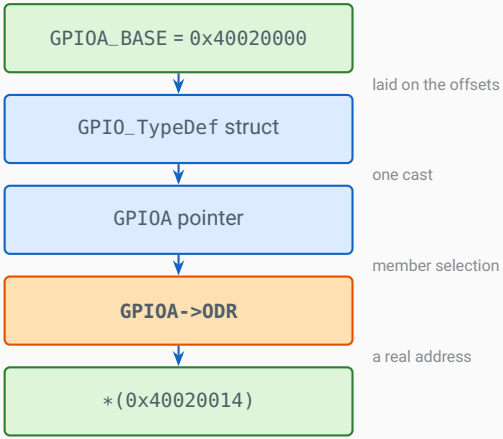
# From Name to Address

```
#define GPIOA_BASE  0x40020000UL

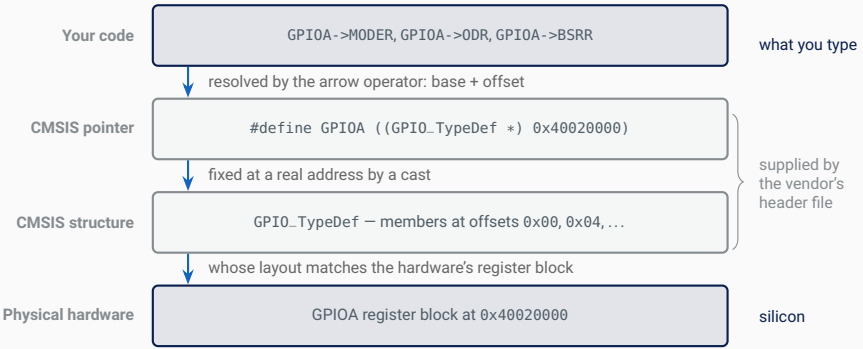
#define GPIOA \
  ((GPIO_TypeDef *) GPIOA_BASE)
```

A cast turns a plain number into a pointer to the struct. After this, the compiler computes every register's address for you.

`ODR` is the sixth member, so its offset is $5 \times 4 = 20 = 0x14$. This gives `0x40020014`, the same address as in Chapter 1.



# The CMSIS Chain Silicon at the bottom, your code at the top




You only need to think about the top and bottom layers: the hardware exists, and your code names its registers. The two middle layers are a struct and a cast. They are written once, in a header file that you never edit.

# Table of Contents


1 Why This Matters

2 What the Compiler Does

3 Why Embedded C Rather Than Assembly

4 The CMSIS Framework

5 The Three Pillars of CMSIS Register Access

6 **The volatile Keyword**

7 Bit Manipulation on Peripheral Registers

8 A Complete Bare-Metal Program

9 What Happens When the STM32 Starts Up

10 Summary

### 1. Caching a value in a register

```
while ((*status & 1) == 0)
    ;   // waits forever without
        // volatile
```

Without `volatile`, the compiler reads the address once, and never checks it again.

### 2. Discarding a “redundant” write

```
*reg = 1;
*reg = 0;   // without volatile the
            // first write is deleted
```

Both writes matter if the address is a pulse output.

## The rule

Every pointer to a hardware register must be `volatile`. So must every variable shared with an interrupt handler.

A compiler may assume that memory only changes when the program changes it. For a status register updated by hardware, that assumption is wrong.

© Muním Zabidi

# Table of Contents


1 Why This Matters

2 What the Compiler Does

3 Why Embedded C Rather Than Assembly

4 The CMSIS Framework

5 The Three Pillars of CMSIS Register Access

6 The volatile Keyword

7 **Bit Manipulation on Peripheral Registers**

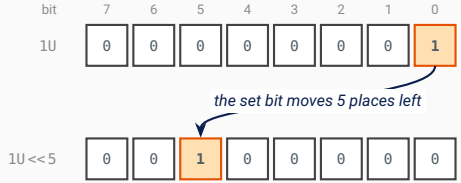
8 A Complete Bare-Metal Program

9 What Happens When the STM32 Starts Up

10 Summary

# Building a Mask: The Shift

Every bit idiom starts with `1U << n`. Begin with `1U`: only bit 0 is set. Shift it left by *n*, and the only set bit is now bit *n*. This value is called a **mask**.



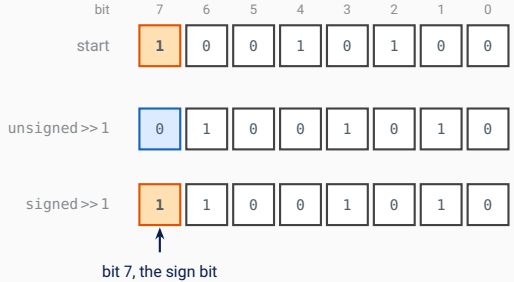

*zeros fill in behind it, so exactly one bit of the mask is set.*

## The shift count *is* the bit number

`1U << 5` selects bit 5, not bit 6. Writing the pin number directly in the shift keeps the code readable, and easy to change for a different pin.

# Why the U Suffix

A plain `1` is signed. Its top bit is the sign bit. C treats signed and unsigned shifts differently, and this difference matters exactly here.

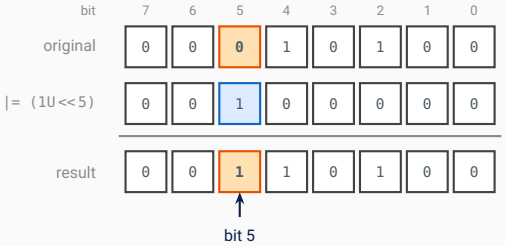



*a **0** is shifted in: the value simply halves.*

*the **sign bit is copied**: a negative value stays negative.*

`1 << 31` is undefined behaviour on a 32-bit `int`. `1U << 31` is fully defined, and gives `0x80000000`. A register is 32 switches, not a signed number. Always write `U`.

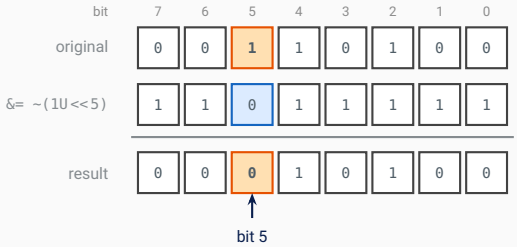
# Setting a Bit: |=




*OR with 1 forces a bit to 1;*
*OR with 0 leaves it alone.*

`GPIOA->ODR |= (1U << 5);` — only the masked bit changes; every other bit is OR-ed with 0 and passes through untouched.

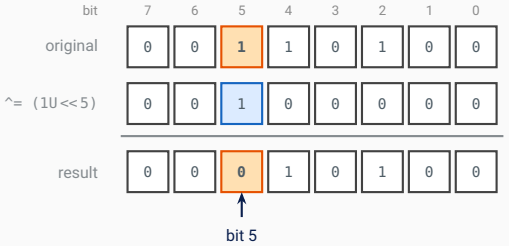
# Clearing a Bit: &= with an Inverted Mask




*AND with 0 forces a bit to 0;*
*AND with 1 leaves it alone.*

`GPIOA->ODR &= ~(1U << 5);` — the tilde makes every bit except bit 5 a 1, so only bit 5 is forced low. The compiler emits `BIC`.

# Toggling a Bit: ^=




*XOR with 1 inverts a bit;*
*XOR with 0 preserves it.*

`GPIOA->ODR ^= (1U << 5);` — the same statement run twice returns the register to its starting value, which is why one line suffices to blink an LED.

# Multi-Bit Fields: Clear, Then Set

OR alone cannot clear a bit. Say `MODER` bits 11:10 read `11`, and you OR in `01`. The result is still `11`. You must clear the field first, before setting it.




*OR alone cannot clear a bit, so a multi-bit field must be zeroed first…*

*…and only then written with the value you want.*

`MODER` bits 11:10 currently read `11` (analog). A student writes:

```
GPIOA->MODER |= (1U << 10);   // want 01
```

**Your prediction**

- Which bits does the OR change?
- What do bits 11:10 hold afterwards?
- Is PA5 now a general-purpose output?

`MODER` bits 11:10 currently read `11` (analog). A student writes:

```
GPIOA->MODER |= (1U << 10);   // want 01
```

**Your prediction**

- Which bits does the OR change?
- What do bits 11:10 hold afterwards?
- Is PA5 now a general-purpose output?

## The answer

OR can only turn bits on. `11` OR `01` is still `11`: PA5 stays analog. Clear the field first: `MODER &= ~(3U << 10);`, then OR in `01`.

# The Four Bit Operations

| Goal | In C | Compiles to |
|---|---|---|
| Set bit *n* | `REG \|= (1U << n);` | `ORR` |
| Clear bit *n* | `REG &= ~(1U << n);` | `BIC` |
| Toggle bit *n* | `REG ^= (1U << n);` | `EOR` |
| Test bit *n* | `if (REG & (1U << n))` | `TST` |

## Read–modify–write

A peripheral register holds settings for many pins. You must keep the bits you are not changing. `|=` and `&=` do this automatically. A plain `=` would wipe out every other setting.

A multi-bit field needs a mask: first clear it with `&= ~(3U << pos)`, then set it with `|= (value << pos)`. The mask is always "all ones for this field."

# Testing Bits and Busy-Wait Polling

### Test a bit

```
if ((GPIOC->IDR & (1U << 13)) == 0) {
    // PC13 is LOW:
    // the button is pressed
}
```

Mask the bit you want. Compare the result with zero.

### Busy-wait on a flag

```
while (!(USART1->SR & (1U << 7))) {
    // wait until the transmit
    // register is empty
}
```

The loop re-reads the register each time. This works only because the register is `volatile`.

# One Register Write, Viewed at Four Levels

| Level | Setting one output bit |
|---|---|
| Physical hardware | The output latch and push-pull driver for pin PA5 |
| Register | `GPIOA_ODR` bit 5, at address `0x40020014` |
| C | `GPIOA->ODR \|= (1U << 5);` |
| Machine code | Three Thumb-2 encodings stored in Flash |

## One row is deliberately missing

Between the C statement and the machine code sits the assembly: the instructions the compiler really emits. The next part of the course fills in that row.

© Munim Zabidi

# Table of Contents


1 Why This Matters

2 What the Compiler Does

3 Why Embedded C Rather Than Assembly

4 The CMSIS Framework

5 The Three Pillars of CMSIS Register Access

6 The volatile Keyword

7 Bit Manipulation on Peripheral Registers

8 **A Complete Bare-Metal Program**

9 What Happens When the STM32 Starts Up

10 Summary

# A Complete Bare-Metal Program

```c
#include "stm32f446xx.h"

int main(void)
{
  // 1. Enable the clock for GPIOA
  RCC->AHB1ENR |= (1U << 0);

  // 2. Set PA5 as an output
  GPIOA->MODER &= ~(3U << (5*2));
  GPIOA->MODER |=  (1U << (5*2));

  // 3. Toggle PA5 forever
  while (1) {
    GPIOA->ODR ^= (1U << 5);
    for (volatile int i = 0;
         i < 100000; i++);
  }
}
```

**Every idea in this chapter**

- Struct-based register access
- Bit masking and shifting
- Read–modify–write
- Clock enable, then configure

**Why `5*2`?** `MODER` gives each pin two bits, so pin 5's bits start at position 10.

**Why `volatile` on `i`?** Without it, the optimiser deletes the whole delay loop.

# The Clock Must Come First

```
RCC->AHB1ENR |= (1U << 0);
```

- Every peripheral starts with its clock disabled, to save power.
- A peripheral with no clock does not respond to writes. They are simply lost.
- `RCC` is the Reset and Clock Control peripheral. GPIOA is bit 0 of `AHB1ENR`.

## The classic first bug

You write the GPIO configuration. The code compiles and runs. But nothing happens. No fault, no error: the writes went nowhere, because the clock was off.

Enable the clock first. Always.

# The Shape of an Embedded Program

### A hosted program (on a PC)

```c
#include <stdio.h>

int main(void) {
    printf("Hello\n");
    return 0;   // back to the OS
}
```

It runs, returns, and the OS cleans up.

### A bare-metal program

```c
int main(void) {
    // setup: runs once
    //   clocks, pins, peripherals

    while (1) {
        // loop: runs forever
        //   read, decide, drive
    }
}
```

No OS. `main` must never return.

## Two consequences of having no operating system

No library to fall back on: `printf` needs a screen and an OS. Output means driving a peripheral yourself. Nothing cleans up: the hardware stays in whatever state you leave it.

*If you know Arduino:* `setup()` is the code above the loop, and `loop()` is its body.

# Table of Contents


1 Why This Matters

2 What the Compiler Does

3 Why Embedded C Rather Than Assembly

4 The CMSIS Framework

5 The Three Pillars of CMSIS Register Access

6 The volatile Keyword

7 Bit Manipulation on Peripheral Registers

8 A Complete Bare-Metal Program

9 **What Happens When the STM32 Starts Up**

10 Summary

# What Happens When the STM32 Starts Up You never call `main`. Who does?




1. **Reset.** The core starts from the 16 MHz internal oscillator. This is why every example assumes 16 MHz.
2. **Read the vector table** at `0x08000000`. Word 0 goes into SP. Word 1 goes into PC. No software has run, yet the CPU has a stack and a start address.
3. **Run `Reset_Handler`.** This startup code copies initialised variables from Flash to SRAM, clears the rest to zero, and calls `SystemInit()`.
4. **Call `main()`.** Now your code runs.

A global such as `int count = 5;` is ready when `main` starts, because startup code copied it. If `main` returned, there would be nowhere to go.

© Muním Zabidi

# Table of Contents


1 Why This Matters

2 What the Compiler Does

3 Why Embedded C Rather Than Assembly

4 The CMSIS Framework

5 The Three Pillars of CMSIS Register Access

6 The volatile Keyword

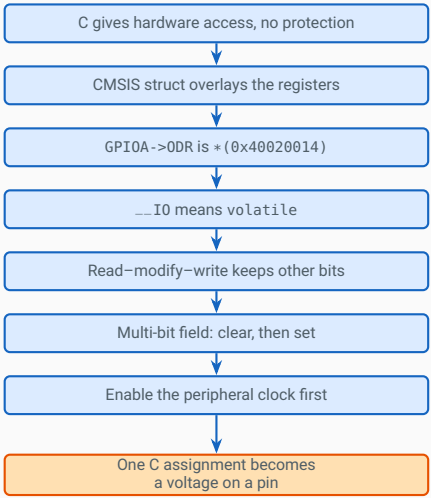
7 Bit Manipulation on Peripheral Registers

8 A Complete Bare-Metal Program

9 What Happens When the STM32 Starts Up

10 **Summary**

# Summary




**Terminology covered**

- CMSIS, device header
- Struct overlay
- `volatile`, `__IO`
- Set, clear, toggle, test
- Peripheral clock enable

**Next: Chapter 5**

*ARM Assembly Fundamentals*

We read what this C compiles to.